



Angular – 21.X.X



Productivity Meets Scalability

Effortless Angular Testing

Angular Component Testing — Complete Walkthrough

A **component test** verifies the behavior of an Angular component: its class logic, template rendering, user interaction, input/output communication, conditional rendering, and interaction with injected services.

We will build a small **Employee component** and test it using **Vitest**, which is the default testing approach in modern Angular applications. The same concepts apply to Jasmine/Karma.

What exactly are we testing?

Suppose we have an Employee Component as:

```
EmployeeComponent
|
+-- Component class
|   +-- employeeName
|   +-- salary
|   +-- incrementSalary()
|
+-- HTML template
    +-- displays employee
    +-- button
    +-- conditional message
```

We want tests such as:

Test	What we verify
Component creation	Component can be instantiated
Initial rendering	Employee name appears
Salary rendering	Salary appears correctly
Button click	Component method executes
State change	UI changes after method execution
Conditional rendering	@if behaves correctly
DOM interaction	User can interact with the template

Follow Steps:

- Create a basic Angular Application

ng new component-testing-demo

- Create employee component



Angular – 21.X.X

Productivity Meets Scalability



ng generate component components\employee

- Modify component class: Employee.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-employee',
  imports: [],
  templateUrl: './employee.html',
  styleUrls: ['./employee.css'],
})
export class Employee {
  employeeName = 'Ravi Kumar';
  department = 'IT';
  salary = 50000;
  incrementSalary(): void {
    this.salary += 5000;
  }
  isHighSalary(): boolean {
    return this.salary >= 75000;
  }
}
```

- Design template: employee.html

```
<div class="employee-card">
  <h2>Employee Details</h2>
  <p>Name: <span data-testid="employee-name">
    {{ employeeName }} </span> </p>
  <p>Department: <span data-testid="department">
    {{ department }} </span> </p>
  <p>Salary: <span data-testid="salary">
    {{ salary }} </span> </p>
  <button data-testid="increment-button"
    (click)="incrementSalary()">Increase Salary</button>
  @if (isHighSalary()) {
    <p data-testid="high-salary">High Salary Employee</p>
  } @else {
    <p data-testid="normal-salary">Normal Salary Employee</p>
  }
</div>
```



Angular – 21.X.X



Productivity Meets Scalability

- Component testing contains an important concept — TestBed, Angular component testing normally starts with it. TestBed creates a testing environment that behaves similarly to Angular's runtime environment.
- Consider following TestSuite and understand

```
import { ComponentFixture, TestBed } from
 '@angular/core/testing';
import { describe, expect, it, beforeEach } from 'vitest';
import { Employee } from './employee';

describe('Employee Component', () => {
  let component: Employee;
  let fixture: ComponentFixture<Employee>;
  // -----
  // Test Setup
  // -----
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [Employee]
    }).compileComponents();
    fixture = TestBed.createComponent(Employee);
    component = fixture.componentInstance;
    // IMPORTANT: Do NOT call fixture.detectChanges() here.
    // Individual tests will call detectChanges()
    // after setting the required component state.
  });
  // -----
  // 1. Component Creation
  // -----
  it('should create the component', () => {
    expect(component).toBeTruthy();
  });
  // -----
  // 2. Component Property Testing
  // -----
  it('should have the correct employee name', () => {
    expect(component.employeeName).toBe('Ravi Kumar');
  });
  it('should have the IT department', () => {
    expect(component.department).toBe('IT');
  });
});
```



Angular – 21.X.X

Productivity Meets Scalability



```
it('should have initial salary of 50000', () => {
  expect(component.salary).toBe(50000);
});
// -----
// 3. Template / DOM Testing
// -----
it('should display employee name', () => {
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const name = element.querySelector('[data-
testid="employee-name"]');
  expect(name?.textContent?.trim()).toBe('Ravi Kumar');
});
it('should display department', () => {
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const department = element.querySelector('[data-
testid="department"]');
  expect(department?.textContent?.trim()).toBe('IT');
});
it('should display salary', () => {
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const salary = element.querySelector('[data-
testid="salary"]');
  expect(salary?.textContent?.trim()).toBe('50000');
});
// -----
// 4. Component Method Testing
// -----
it('should increase salary by 5000', () => {
  component.incrementSalary();
  expect(component.salary).toBe(55000);
});
// -----
// 5. Button Click Testing
// -----
it('should increase salary when button is clicked', () => {
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
```



Angular – 21.X.X

Productivity Meets Scalability



```
const button = element.querySelector('[data-
testid="increment-button"]') as HTMLButtonElement;
button.click();
fixture.detectChanges();
expect(component.salary).toBe(55000);
});
// -----
// 6. Verify UI after Button Click
// -----
it('should update salary in the UI after button click', ()
=> {
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const button = element.querySelector('[data-
testid="increment-button"]') as HTMLButtonElement;
  button.click();
  fixture.detectChanges();
  const salary = element.querySelector('[data-
testid="salary"]');
  expect(salary?.textContent?.trim()).toBe('55000');
});
// -----
// 7. Testing @if - False Condition
// -----
it('should display normal salary message initially', () => {
  // salary is 50000 before first change detection
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const normalSalary = element.querySelector('[data-
testid="normal-salary"]');
  expect(normalSalary).not.toBeNull();
});
// -----
// 8. Testing @if - True Condition
// -----
it('should display high salary message when salary is 75000
or more', () => {
  // IMPORTANT:
  // Set the state BEFORE the first detectChanges()
```



Angular – 21.X.X

Productivity Meets Scalability



```
component.salary = 75000;
fixture.detectChanges();
const element: HTMLElement = fixture.nativeElement;
const highSalary = element.querySelector('[data-
testid="high-salary"]');
expect(highSalary).not.toBeNull();
});
// -----
// 9. Testing @if - Opposite Condition
// -----
it('should not display normal salary message for high salary
employee', () => {
  // Set state before first change detection
  component.salary = 75000;
  fixture.detectChanges();
  const element: HTMLElement = fixture.nativeElement;
  const normalSalary = element.querySelector('[data-
testid="normal-salary"]');
  expect(normalSalary).toBeNull();
});
});
```

- After running test you should now get all tests passing:

ng test

```
✓ component-testing-demo src/app/components/employee/employee.spec.ts (13 tests) 115ms
✓ Employee Component (13)
  ✓ should create the component 44ms
  ✓ should have the correct employee name 9ms
  ✓ should have the IT department 6ms
  ✓ should have initial salary of 50000 7ms
  ✓ should display employee name 13ms
  ✓ should display department 5ms
  ✓ should display salary 4ms
  ✓ should increase salary by 5000 3ms
  ✓ should increase salary when button is clicked 6ms
  ✓ should update salary in the UI after button click 5ms
  ✓ should display normal salary message initially 4ms
  ✓ should display high salary message when salary is 75000 or more 3ms
  ✓ should not display normal salary message for high salary employee 3ms
✓ component-testing-demo src/app/app.spec.ts (2 tests) 121ms
✓ App (2)
  ✓ should create the app 98ms
  ✓ should render title 21ms

Test Files 2 passed (2)
Tests 15 passed (15)
Start at 09:35:34
Duration 865ms
```

❄️❄️ Catch the bugs before they change your Signals. ❄️❄️